



AA

Advanced Portfolio Web Application

A comprehensive personal web application showcasing API integration, state management, and modern front-end development.

Abdulrahman Alomar • Student ID: 202263820
SWE-363 — Assignment 4 • Spring 2026

OVERVIEW

Presentation Agenda



Introduction

Project overview, objectives, and personal motivation



Technical Demo

Live demonstration of key features and architecture



Deep Dive

Challenges, solutions, and lessons learned



Conclusion

Outcomes, AI integration, and future plans

INTRODUCTION

Project Overview & Motivation

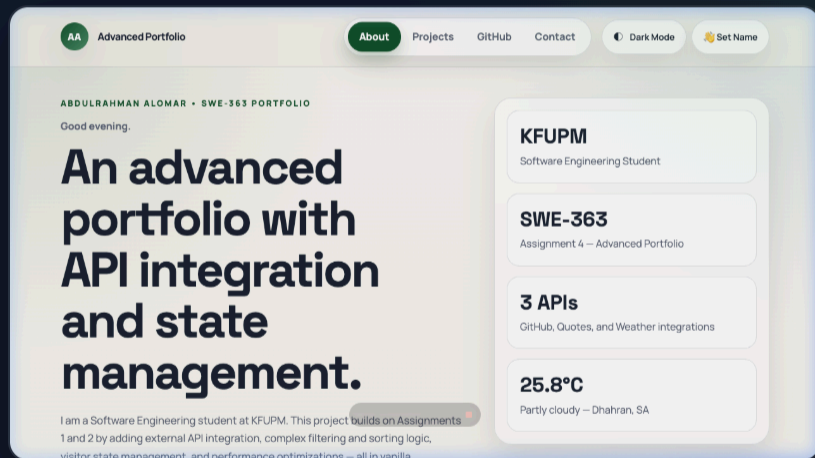
What I Built

- ▶ A **single-page portfolio** application built from scratch
- ▶ **4 tab sections**: About, Projects, GitHub, Contact
- ▶ Vanilla HTML, CSS, and JavaScript — **no frameworks**
- ▶ Live deployment, fully responsive across all devices

HTML5

CSS3

ES6+



LANDING PAGE — LIGHT MODE

Architecture & Tech Stack

File Structure

- ▶ `index.html` — Semantic structure with ARIA attributes
- ▶ `css/styles.css` — 1,049 lines, CSS custom properties for theming
- ▶ `js/script.js` — 666 lines, organized into logical sections
- ▶ `docs/` — AI report & technical documentation

Design Principles

- ▶ No frameworks — pure vanilla implementation
- ▶ CSS custom properties for instant theming
- ▶ Progressive enhancement — works without JS for base content
- ▶ Mobile-first responsive design

index.html



styles.css



script.js



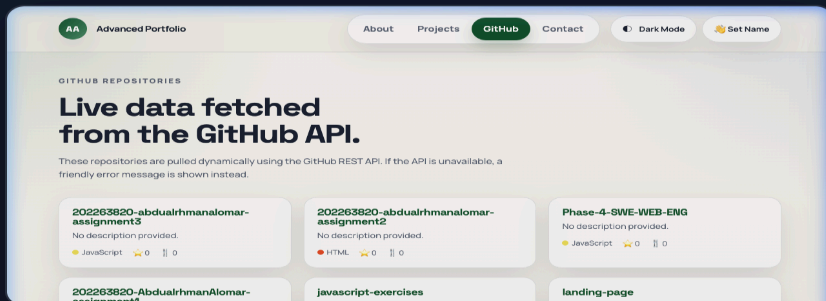
External APIs



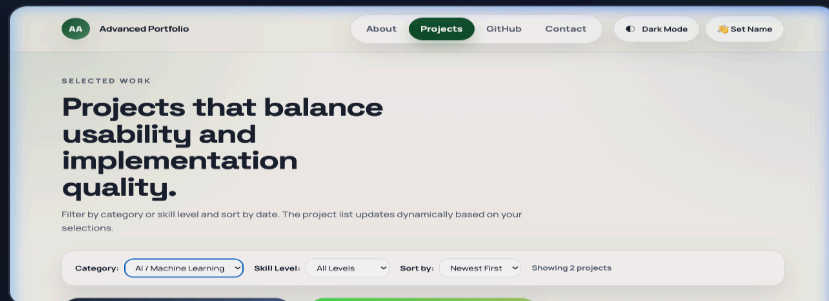
localStorage

TECHNICAL DEMONSTRATION

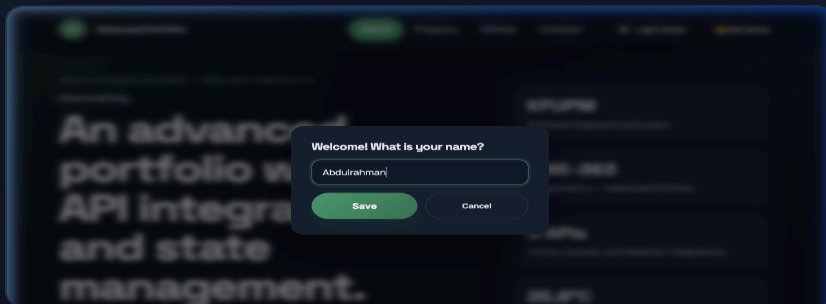
Key Features — Live Screenshots



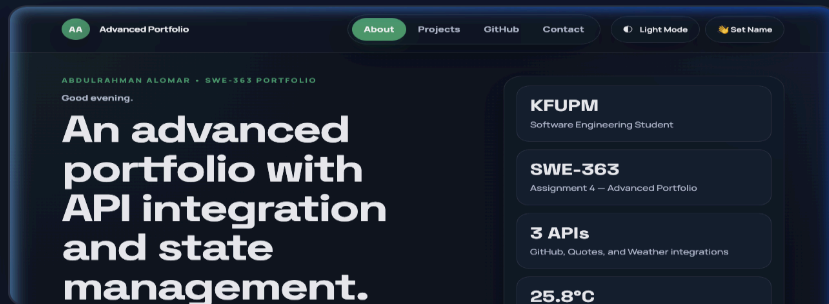
 GITHUB API — LIVE REPOS



 FILTER & SORT — AI CATEGORY



 VISITOR STATE — LOGIN MODAL



 DARK MODE THEME

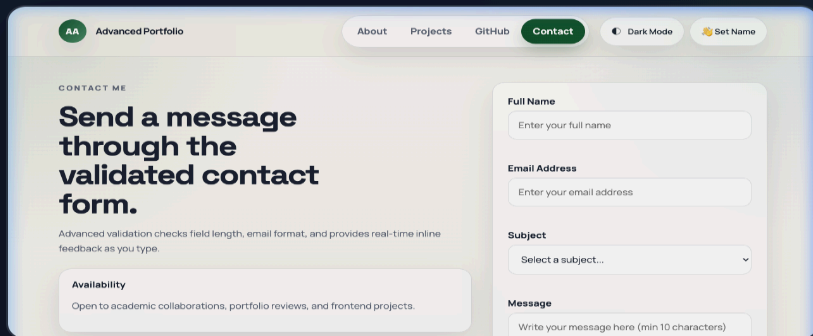
API Integration Highlights

How APIs Work

1. User opens tab / loads page
2. `fetch()` with `async/await`
3. Parse JSON response
4. Render DOM elements dynamically
5. Handle errors gracefully

Error Handling

- ▶ **GitHub** — Error card with Retry button
- ▶ **Quotes** — Falls back to local array
- ▶ **Weather** — "Weather unavailable"



The screenshot shows a web application titled 'Advanced Portfolio' with a navigation bar containing 'About', 'Projects', 'GitHub', 'Contact', 'Dark Mode', and 'Set Name'. The 'Contact' tab is active. The main content area is titled 'CONTACT ME' and features a heading 'Send a message through the validated contact form.' Below the heading is a subtext: 'Advanced validation checks field length, email format, and provides real-time inline feedback as you type.' There are two boxes: 'Availability' (Open to academic collaborations, portfolio reviews, and frontend projects.) and a form with four fields: 'Full Name' (text input), 'Email Address' (text input), 'Subject' (dropdown menu), and 'Message' (text area with a note 'Write your message here (min 10 characters)').

CONTACT FORM — VALIDATION

Security

- ▶ `escapeHTML()` prevents XSS on API data
- ▶ `encodeURIComponent()` for safe external links
- ▶ `rel="noopener noreferrer"` on outbound links

Challenges & Solutions

✓ Resolved

- ▶ **CORS issues** — Replaced key-required APIs with free CORS-friendly alternatives
- ▶ **XSS vulnerabilities** — Built `escapeHTML()` utility and input sanitization
- ▶ **Theme flicker on load** — Read `localStorage` before first paint
- ▶ **Filter state sync** — Chained `.filter()` + `.sort()` pipeline

🔮 Future Work

- ▶ **Backend integration** — Contact form currently client-side only
- ▶ **User authentication** — Real OAuth-based login
- ▶ **CMS for projects** — Dynamic content management instead of hardcoded data
- ▶ **Offline support** — Service workers for PWA capability

AI Tools & Responsible Use

Tools Used

GITHUB COPILOT

CLAUDE

CHATGPT

- ▶ **Code generation** — API functions, filter logic
- ▶ **Debugging** — DOM references, event binding
- ▶ **Code review** — Security, organization
- ▶ **Documentation** — README, technical docs

Responsible Use

- ▶ All AI output was **reviewed and understood**
- ▶ **Replaced** unsuitable API suggestions (keys, CORS)
- ▶ **Removed** over-engineered patterns (debouncing, service workers)
- ▶ **Rewrote** documentation to match actual implementation
- ▶ AI as **assistant**, not replacement for learning

Lessons Learned



Architecture Matters

Organizing JS into clear sections (DOM refs, features, events, init) makes 666 lines manageable.



Security First

Always escape dynamic content from external APIs before inserting into the DOM.



Graceful Degradation

Every API call needs a fallback. Users should never see a broken interface.



Performance

Lazy loading, deferred scripts, and CSS variables keep the app fast and responsive.



AA

Thank You

A production-ready portfolio showcasing API integration, state management, responsive design, and security best practices — built entirely with vanilla web technologies.

6 PROJECTS

3 APIS

DARK / LIGHT THEME

FULLY RESPONSIVE

Abdulrahman Alomar • 202263820

SWE-363 • Spring 2026